INTELLIHIRE / DELIVERY BLUEPRINT

Frontend-Backend Integration Plan

A codebase-specific contract and test plan for connecting the existing React screens to the Django API across Modules 1-13.

SCOPE	REPOSITORY BASELINE	DELIVERY RULE
API integration only; no application edits or integration work in this phase.	Frontend remains demo-backed; backend exposes the module API surface.	One module at a time; test, report status, obtain approval before advancing.

Prepared 13 November 2026 | Current checkout reviewed against frontend source, backend URL configuration, serializers, permissions and live OpenAPI schema.

Status language: SCOPED COMPLETE means all non-AI work in the stated scope is implemented and verified. AI GATED means the workflow must not be simulated or reported complete until hooks, provider access and output tests are ready.

Executive Summary

The current checkout is not API-integrated for Modules 1-4. The shared apiClient is present but no feature service consumes it; authApi and auth state are mock/localStorage-backed. Candidate, company, admin, audit and community stores also persist seeded/demo entities. The backend, by contrast, mounts APIs for all thirteen modules. Integration should therefore begin at identity and proceed in dependency order, not at Module 5.

> Scope decision: Modules 6, 9 and 11 are included for non-AI API behavior. Modules 7 and 8 include invitation/session management contracts but their generated interview content, answer scoring and AI reports remain gated. Module 13 AI settings, prompt management and usage controls are deferred.

The integration sequence

01 Auth	**02 Company**	**03 Jobs**	**04 Applications**	**05 Notifications**	**06 CV**
07/08 Interviews	**09 Evaluation**	**10 Tracking**	**11 Mock**	**12 Communities**	**13 Admin**

How to use this document

- Treat each module page as the approved work boundary: screen mapping, endpoint contract, persisted data, permission rules and verification gate.
- Consult the backend OpenAPI schema for typed payload definitions; where it reports generic objects, the serializer/view and contract tests are the source of truth.
- Do not advance to the next module until the current module has a written test/status report and explicit user approval.

Verified baseline

Area	Observed current state	Integration consequence
Frontend API	apiClient uses /api, reads access token from localStorage, always JSON-encodes body and always parses JSON.	Add cookie-aware refresh, multipart/binary/204 handling, normalized API errors and feature services before broad endpoint work.
Frontend data	Auth and feature stores contain demo accounts/IDs and persisted seeded objects.	Cut over to UUIDs; clear or version stale per-user storage and do not treat demo IDs as backend identities.
Backend contract	Django URL config mounts all modules; generated OpenAPI has 160+ route paths and shared pagination/auth behavior.	Use schema plus serializer/view tests; generic response objects require explicit contract assertions.
FE tests	Vite/React app has no Vitest/RTL/MSW test script or Playwright workflow setup.	Add the test harness as integration foundation; run it with each module, not only at the end.

Cross-Cutting Integration Contract

Implement shared transport and state behavior before endpoint-by-endpoint work. These decisions reduce inconsistent authentication, file handling and pagination across the 13 modules.

Authentication and deployment

- Use same-origin /api in browser. Add a Vite dev proxy to Django and a production reverse-proxy route; avoid browser CORS dependence for refresh cookies.
- Hold short-lived access JWT in memory only. Put rotating refresh JWT in HttpOnly; SameSite=Lax cookie, scoped to /api/auth/; set Secure in production. Refresh returns a new access JWT and rotates the cookie; logout blacklists and clears it.
- Frontend sends credentials: include and never reads, persists or logs refresh JWT. Preserve JSON refresh-token body on backend temporarily for compatibility; frontend uses cookie flow only.
- Cookie-backed unsafe auth actions require verified Origin and CSRF protection. Document trusted origins, cookie path/domain, reverse-proxy TLS and local HTTPS behavior.
- On app boot, attempt refresh then GET /api/auth/me/. Coalesce concurrent refresh calls, retry a failed request at most once, and clear in-memory user state on terminal 401.

API client and transport

- Create a typed ApiError carrying status, detail, code and field errors; backend errors may be {detail, code} or serializer field maps. Preserve the original response for diagnostics without exposing secrets.
- Set Content-Type only for JSON. FormData uploads must let fetch set the multipart boundary; binary exports/downloads use Blob and content-disposition filename; 204/205 responses do not call res.json().
- Make query parameter serialization explicit for page/page_size, repeated IDs, booleans, dates, ordering and search. Parse paginated {count,next,previous,results} into a shared UI adapter.
- Centralize request cancellation, timeout, request IDs and structured logging; redact auth headers, private tokens, CV text and PII.

Client state, identity and authorization

- Use backend UUIDs as every entity key. Replace demo IDs only after a successful server read; version local caches and clear user-specific persisted Zustand keys at login/account switch.
- Do not hydrate stale local demo objects into a real account. Keep only safe preferences that have no backend counterpart, label those as local-only, and avoid persistence claims.
- Backend role checks remain authoritative. Frontend route guards and hidden controls improve UX only; test 401, 403, 404 and 409 with stale state and direct URLs.
- Model loading, empty, success, validation error, server error, retry and stale/conflict states for every view. Do not represent an API error as an empty list or missing AI result as score zero.

Pagination, files and state transitions

Contract	Required frontend behavior
Pagination	Keep server page and page_size in URL/state; handle next/previous links and count; reset page when filters change; preserve filters on back navigation.
Multipart	Resume/application/community attachments use FormData with per-field validation; show progress, size/type errors, retry and server persistence after reload.
Binary	Resume, evaluation PDF and exports use Blob; parse content type/name, revoke object URLs, handle 401/403/404 and never JSON-parse file bytes.
Mutations	Disable duplicate submits; use server state as final truth; reconcile after 409/validation errors; do not optimistically create irreversible HR/admin outcomes.
Tokens	Interview/tracking/invitation tokens are bearer credentials. Avoid localStorage, analytics, console output, screenshots and referrer propagation.

> Backend baseline to preserve: StatusAwareJWTAuthentication; authenticated-by-default APIs; 15-minute access JWT; 7-day rotating refresh; blacklist after rotation; default page size 20; DRF pagination envelope. Confirm actual settings in the environment before rollout.

MODULE 01 / SCOPED INTEGRATION

Authentication and candidate profile

Replace mock login, registration and profile persistence with server-backed identity, role routing and candidate data. The selected auth contract uses an in-memory access token and refresh cookie.

Existing screen and workflow mapping

LoginPage, RegisterPage, CandidateProfile, CandidateSettings, protected route guards, AdminUsers user actions. Preserve the existing screens; make the APIs their source of truth.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
POST `/api/auth/register/candidate/`	JSON: email, password, name/profile fields as defined by CandidateRegistration; validate password and required identity fields.	Creates User + CandidateProfile; response returns safe user/account state. Email verification is a separate transition.	Public. Handle duplicate email (409/400), field validation (400), and unverified state without creating a fake signed-in session.
POST `/api/auth/login/ | /api/auth/verify-email/ | /api/auth/resend-verification/`	Login JSON: email + password. Verification/resend payload uses the email/token contract in serializers.	Login returns access JWT + user summary and sets HttpOnly refresh cookie. Verification updates user verified state; resend is rate-limited.	Public. Invalid credentials must not reveal account existence; show actionable validation and throttling states.
POST `/api/auth/token/refresh/ | /api/auth/logout/`	Frontend sends cookie credentials; refresh body omitted. Legacy JSON refresh token remains accepted temporarily. Logout clears cookie and blacklists token.	Refresh rotates refresh cookie and returns a new short-lived access token. Logout is idempotent from the UI perspective.	Cookie-backed unsafe actions require Origin/CSRF checks. Refresh rotation and blacklist enabled; refresh once after concurrent 401s.
GET/PATCH `/api/auth/me/ | /api/auth/profile/`	PATCH safe user/profile fields: name, phone, headline, location, bio, target role, experience level, links, skills, discoverability.	User identity/role plus CandidateProfile; profile includes education, experience, resume metadata and updated_at.	Authenticated owner. Reject role escalation and company-scoped profile edits. Treat 401 as refresh attempt then sign out.
GET/PUT `/api/auth/profile/resume/`	Multipart file upload; do not JSON-stringify FormData. Enforce allowed file type/size and replace semantics.	Resume metadata and/or file response; retain original name and updated timestamp. Binary retrieval must use Blob handling if returned.	Candidate owner. Test invalid type, size, empty file, storage failure, and download content disposition.
CRUD `/api/auth/profile/education/ | /experience/`	JSON records: institution/degree/field/date fields and experience company/title/description/date fields; PATCH/DELETE by record ID.	Nested profile collections persist through Education and Experience rows linked to CandidateProfile.	Candidate owner only. Handle stale IDs and invalid dates; refresh profile after successful mutation.
POST `/api/auth/password/forgot/ | /reset/ | /change/`	Email-only forgot request; reset token + password; change current/new password.	Neutral forgot response; reset/change response is an operation result, never a password value.	Forgot flow is anti-enumerating and rate-limited. Clear access state after password change if policy requires.

Data relationships

User (UUID primary key, role, verification/status); CandidateProfile one-to-one User; Education and Experience foreign keys to profile. Frontend demo keys such as usr_c1 must never be sent as identifiers.

Required verification

Valid/invalid registration and login; verified/unverified routing; refresh rotation, parallel 401 refresh and logout blacklist; profile ownership; education/experience CRUD; multipart resume MIME/size; password reset anti-enumeration; role permission and persistence.

Acceptance gate: No localStorage token remains. Auth boot works after reload through refresh cookie; route role derives from /me; profile changes survive a fresh browser context. Secure cookie and CSRF/origin checks pass in test and production-like config.

MODULE 02 / SCOPED INTEGRATION

Company, onboarding and team access

Connect company onboarding, approval state, dashboard, staff management and invitations to persisted company membership rather than demo stores.

Existing screen and workflow mapping

CompanyOnboarding, CompanyPending, CompanyDashboard, CompanySettings, CompanyTeam, company profile and the existing AdminCompanies review screens. Public invitation acceptance is an identified missing route.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
POST `/api/auth/register/company/ | /api/auth/register/company/resubmit/`	Company registration JSON: organization identity, website, contact and verification details; resubmit fields are the editable onboarding payload.	Company + initial admin membership; approval/pending state and safe company profile returned. Resubmit updates review state.	Public registration; resubmit requires matching company owner. Duplicate/invalid org details surface field errors.
GET/PATCH `/api/company/profile/ | GET /api/company/dashboard/`	Profile patch for company fields; dashboard is read-only.	Company profile, approval state, member context; dashboard aggregates job/application counts and recent items.	Active approved company member. Viewer can read; profile write requires company admin or HR manager.
GET/PATCH `/api/company/members/ | /{member_id}/`	List filters/page; patch member role or active state as supported by serializer.	Paginated members with user identity, role and membership status. Membership rows bind User to Company.	Company admin/HR manager may manage roles; enforce last-admin and self-demotion constraints in backend response.
GET/POST/POST `/api/company/invitations/ | /{invitation_id}/resend/ | /revoke/`	Invite email + allowed role; resend/revoke are action posts.	Invitation state, expiry, inviter and email; actions return updated invitation state.	Company admin/HR manager. Duplicate active invite, expired and already accepted states are explicit.
GET/POST `/api/public/invitations/{token}/ | /accept/`	Opaque invitation token; accept requires authenticated account or creates/links account per backend contract.	Invitation preview and accepted membership result.	Public token is a credential: do not log it. Acceptance screen is absent today and is a gap, not a new design scope.
GET/POST `/api/platform-admin/companies/ | /{company_id}/approve|reject|suspend|reactivate/`	Paginated review filters; action reason for reject/suspend where required.	Company review/detail and onboarding evidence; action returns updated approval/status and audit event.	Platform admin only. Model state transition errors (409); company admin must not self-approve.

Data relationships

Company; CompanyMember (User and Company foreign keys, role/status); StaffInvitation (company, inviter, email, role, token hash/expiry). Platform decisions append activity/audit records.

Required verification

Registration and pending state, owner resubmission, member list/write role matrix, invitation create/resend/revoke/expiry/acceptance token behavior, company isolation, admin approval transitions, dashboard data from DB.

Acceptance gate: Company routes hydrate against real company UUID and membership; company isolation and viewer read-only behavior are verified. Missing public invitation acceptance is recorded as a required workflow gap before claiming end-to-end onboarding.

MODULE 03 / SCOPED INTEGRATION

Job posting and application form builder

Replace local job CRUD with persisted job lifecycle, searchable lists, custom form fields and publish state. The public form route exists in the backend; no apply UI is included in this integration scope.

Existing screen and workflow mapping

CompanyJobs, CompanyJobCreate, CompanyJobPreview, job editor, department selectors and existing admin job monitor. Keep preview distinct from published public form.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET/POST `/api/jobs/ | GET/PATCH/DELETE /api/jobs/{job_id}/`	Job payload includes title, department, description, responsibilities, employment_type, work_mode, location, skills, experience/education, salary, openings, deadline and status fields.	JobPosting with company ownership, UUID, lifecycle status and timestamps; list is paginated.	Active company admin/HR manager/recruiter for writes; company viewer read-only. Reject writes on closed/deleted jobs.
GET `/api/jobs/departments/`	Optional search or list params per schema.	Department options for editor; use backend identifiers, not labels as foreign keys.	Authenticated company context; show empty state when no options.
GET/POST/PATCH/DELETE `/api/jobs/{job_id}/fields/ | /fields/{field_id}/ | POST /fields/reorder/`	Custom field label, key, type, required, options/order. Reorder uses ordered field IDs.	ApplicationField records scoped to JobPosting; ordering and validation metadata returned.	Write roles only. Reject duplicate keys, unsupported field types and cross-job field IDs.
GET/POST `/api/jobs/{job_id}/form-preview/ | /api/public/jobs/{form_slug}/`	Preview uses saved draft; public read uses unguessable/published slug.	Render-ready job and form field definitions. Public route exposes only safe published fields.	Preview company member; public published form is anonymous. Draft/closed visibility must match backend policy.
POST `/api/jobs/{job_id}/publish/ | /close/ | /reopen/ | /duplicate/`	Action request; duplicate uses source ID and server rules. Publish validates required job fields/deadline.	Updated job state or new draft; persist all transitions.	Admin/HR manager/recruiter. Stale transitions return conflict; UI must refresh, not optimistically assume success.
GET `/api/platform-admin/jobs/ | /{job_id}/`	Search/status/company/page filters where supported.	Read-only platform view of job and owning company.	Platform admin only. No admin job mutation is implied by a monitor screen.

Data relationships

JobPosting belongs to Company; Department and ApplicationField are referenced from a job. Public form slug is distinct from internal job UUID.

Required verification

Create/edit/delete and publish/close/reopen state machine; custom field reorder and validation; list filter/page; company isolation; preview vs public visibility; duplicate semantics; persisted refresh; viewer permissions.

Acceptance gate: Every job visible after reload comes from API. Publish is blocked until backend validation passes; stale status errors reconcile from a fresh GET. Public apply remains separately flagged in Module 4.

MODULE 04 / SCOPED INTEGRATION

Applications and employer review

Connect employer application list/detail, status changes, notes, resume and custom answer files. The public apply endpoint exists but the candidate apply screen is absent.

Existing screen and workflow mapping

CompanyCandidates, CompanyAllCandidates, CompanyCandidateProfile and application detail/drawer. CandidateApplications is a tracking view, not public job application submission.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/applications/ | /api/jobs/{job_id}/applications/`	Filters include education_level, job, min_cv_score, ordering, page/page_size, search, status and submitted date bounds.	Paginated {count,next,previous,results}; rows include UUID/reference/name/email/phone/education/status/submitted_at/has_cv and nullable screening, technical, evaluation fields.	Company member scoped to owned jobs; viewers read-only. Keep nullable module 6-9 data distinct from zero score.
GET `/api/applications/{application_id}/`	Application UUID path.	Detail includes candidate/application data, answers, status history, notes and module score summaries when available.	Company membership + application job ownership. A foreign UUID must not disclose candidate data.
PATCH `/api/applications/{application_id}/status/`	Status transition plus optional reason/note per serializer.	Updated status and history entry; side effects may create notification/activity events.	Admin/HR manager/recruiter. Invalid or stale transition returns 400/409 and refreshes state.
GET/POST `/api/applications/{application_id}/notes/`	Create note text; list read-only.	Notes with author, timestamp and content. Notes are internal, not candidate-safe.	Company access only; validate ownership and length; never expose notes in Module 10 tracking.
GET `/api/applications/{application_id}/resume/ | /files/{field_key}/`	No JSON body. Resolve server file URL or return binary response.	Resume/application upload bytes with MIME and content-disposition metadata.	Company access to owning application. Use Blob/download path and handle missing file/403/404.
GET `/api/jobs/{job_id}/applications/export/`	Same filters as list where supported.	CSV/XLSX or backend-declared binary export; content type and filename are authoritative.	Company access. Test export filter parity and avoid routing binary body through JSON parser.
POST `/api/public/jobs/{form_slug}/apply/`	Multipart candidate identity, resume and dynamic field answers; exact accepted fields follow public application serializer.	Application + answer rows linked to JobPosting; reference and submission state returned.	Anonymous public endpoint; rate limit and validate published slug, file constraints and required answers. UI is absent; no new screen designed here.

Data relationships

Application belongs to JobPosting and candidate User; answer rows reference ApplicationField; resume and custom attachments use file fields; status history and private notes are related records.

Required verification

Application list pagination/filter parity; UUID ownership; status transition/duplicate requests; notes visibility; multipart resume and custom file validation; export bytes; application DB graph; public endpoint rate limit/anti-abuse even though UI gap remains.

Acceptance gate: Company list/detail and status updates persist; binary resume/export work; scores from downstream modules render as unavailable until sourced. End-to-end candidate apply remains blocked by missing public apply UI.

MODULE 05 / SCOPED INTEGRATION

Notifications and foundation

Turn the existing candidate notification bell into an API-backed, role-aware notification center and mount the shared control in company and admin headers where appropriate.

Existing screen and workflow mapping

HeaderMenus notification popover; candidate layout already mounts it. Company/admin bell placement is incomplete. Candidate/company notification preferences remain local and have no preference API.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/notifications/`	Paginated; `?unread=true|false`, page and page_size. Use results/count/next/previous envelope.	Notification fields: id, kind, title, body, link, payload, read, read_at, created_at.	Authenticated user; server scopes to own User FK. Keep ordering deterministic and unread count consistent.
GET `/api/notifications/unread-count/`	No request body.	Unread count integer/object; use as badge count and reconcile after mutations.	Authenticated user only. Loading/error state must not imply zero unread.
POST `/api/notifications/{notification_id}/read/ | /read-all/`	Single notification UUID or no body for all.	Updated read_at/read state; action is idempotent.	Authenticated user owner only. Foreign notification behaves as not found/forbidden per backend; prevent double count decrement.

Data relationships

Notification foreign key to User; related business actions may emit ActivityEvent/SystemAlert or email records. Notification preferences are not an API contract today.

Required verification

Per-role own-data isolation; unread filter/count; read-one/read-all idempotence; pagination; deep links to authorized routes; notification creation triggered by business events; race-safe badge reconciliation.

Acceptance gate: Bell shows API count/list and handles loading, empty, stale, mark-read and route states. Preference toggles remain labelled local-only until a backend API is built; do not imply persistence.

MODULE 06 / SCOPED COMPLETE CANDIDATE | AI GATED ACTIONS

CV analysis and shortlisting

Integrate stored configuration, status/progress, ranking and shortlist threshold now. CV extraction, embeddings, score generation, reruns and semantic explanations remain blocked on working AI hooks/provider access.

Existing screen and workflow mapping

CompanyCVAnalysis, candidate score summaries and shortlist controls. Build against backend states and stored scores; do not fabricate explanations or scores.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET/PUT `/api/jobs/{job_id}/screening-config/`	Config fields: auto_analyse, shortlist_threshold, auto_shortlist, invite_on_shortlist, weight_skills, weight_experience, weight_education, weight_semantic.	JobScreeningConfig one-to-one with JobPosting; returns updated_at and warnings.	Company write roles depend on endpoint permission (settings restricted to admin/HR manager); validate normalized weights and threshold.
GET `/api/jobs/{job_id}/cv-screening/`	Job UUID; optional status/progress filters if schema supports them.	Counts and run/progress status for applications; no score should be inferred when absent.	Company access for owned job. Poll only while run is active; stop on terminal/error state.
GET `/api/jobs/{job_id}/ranking/`	Filters: min_score, status, analysis_status, page/page_size.	Paginated ranked applications with stored CVAnalysis/CVMatch values and nullable explanation/status.	Company access. Stable tie ordering; never convert null score to 0.
POST `/api/jobs/{job_id}/cv-screening/run/ | /api/applications/{application_id}/cv-analysis/rerun/`	Run mode: missing, rescore or full; rerun targets an application.	Accepted task/run state with job/application association; asynchronous progress and errors are polled from screening GET.	AI GATED. UI can expose disabled/ready states only after provider hooks and fake-AI tests. Do not report a completed score without persisted result.
GET `/api/applications/{application_id}/cv-analysis/`	Application UUID.	Persisted analysis and match breakdown linked one-to-one to Application.	Company owner. Returned analysis may be empty/unavailable; distinguish pending, failed and not run.
POST `/api/jobs/{job_id}/shortlist/apply-threshold/`	threshold + dry_run; optional confirmation for apply.	Dry-run candidate IDs/count, or persisted application shortlist transitions and event records.	Write roles; confirm whether operation is atomic and how notifications are emitted. Reconcile after response.

Data relationships

JobScreeningConfig one-to-one JobPosting; CVAnalysis/CVMatch one-to-one Application; asynchronous tasks associate run state to job/applications. AI uses llm.generate_json and llm.embed hooks.

Required verification

Config validation/permissions; progress transitions; rank filters/pagination; threshold dry-run vs apply DB effects; empty/failed AI result; fake AI deterministic extraction/embedding, retry/idempotency and concurrent run control before enabling gated actions.

Acceptance gate: Non-AI screens, config, existing scores and threshold behavior may be scoped complete after module tests. Run/rerun and generated score/explanation remain AI GATED until hook/provider readiness and explicit result validation; no full Module 6 completion claim yet.

MODULE 07 / SCOPED COMPLETE CANDIDATE | AI GATED SESSION

Invited screening interviews

Integrate employer invitation lifecycle, list/detail, resend/extend/cancel and expiry state. Candidate session start, question generation, answer scoring and AI evaluation remain gated.

Existing screen and workflow mapping

CompanyInterviews and candidate CandidateInvitedInterview. Existing Candidate InterviewSetup/Room/Report are local practice flows and are not the invited screening contract.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET/POST `/api/jobs/{job_id}/interviews/ | /api/applications/{application_id}/interviews/`	Create invitation payload uses application/candidate, kind, schedule/expiry and interview settings defined by InterviewInvite serializer.	InterviewSession rows linked to Application and candidate User; list returns kind, state, token/link metadata and timestamps.	Company write roles for invitation; ownership scoped by job/application. Token itself must be treated as secret.
GET `/api/interviews/{session_id}/`	Session UUID.	Employer session detail and lifecycle, turns/results only when present.	Company owner; candidate-facing data filtered. Do not return private employer notes to candidate.
POST `/api/interviews/{session_id}/resend|extend|cancel|reevaluate/`	Resend/extend action parameters per serializer; cancel optional reason; reevaluate action.	Updated invitation/session state and audit/notification effect.	Company write roles; invalid terminal transitions return 409. Reevaluation is AI GATED.
GET `/api/public/interviews/{token}/`	Opaque invitation token in path; no application UUID in URL.	Minimal invite/session context required to begin, expiry and status.	Anonymous token access is rate-limited and non-enumerable. Prevent token leakage to logs/referrers.
POST `/api/public/interviews/{token}/start|answer|finish/`	Start; answer body uses question/answer turn contract; finish finalizes session.	InterviewTurn records and final score/report state.	AI GATED for question creation, answer scoring and generated report. Do not show bank fallback as AI-complete; retry/duplicate answer behavior must be deterministic.

Data relationships

InterviewSession belongs to Application and candidate User; InterviewTurn belongs to session. Kinds include screening and technical; Module 8 uses the same session family.

Required verification

Invitation owner/role, resend/extend/cancel state matrix, token expiry/revocation, candidate session isolation, turn ordering/idempotency, finish rules, persisted report; fake-AI output/timeout/error and no empty-score success before enabling.

Acceptance gate: Employer invitation management may reach scoped complete after DB and security checks. Session start/answer/finish plus AI-generated questions/scores/reports stays gated until ai_hooks.py and provider access are ready.

MODULE 08 / SCOPED COMPLETE CANDIDATE | AI GATED SESSION

Technical interview

This module has no separate URL configuration. It is a technical-kind workflow on Module 7 InterviewSession/InterviewTurn endpoints and schemas.

Existing screen and workflow mapping

Reuse CompanyInterviews and CandidateInvitedInterview presentation where practical; label the interview kind and use technical competency/result fields only when the backend returns them.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
Shared `/api/jobs/{job_id}/interviews/ | /api/applications/{application_id}/interviews/`	Create/list with kind=technical plus the shared scheduling, expiry and candidate fields. No Module 8-specific route.	Shared InterviewSession with technical kind and linked application/user; result fields may be absent until evaluated.	Same company ownership and write-role rules as Module 7. Do not invent a parallel API client contract.
Shared `/api/interviews/{session_id}/ | /resend|extend|cancel|reevaluate/`	Use shared session UUID and lifecycle actions.	Shared session state, turns and persisted technical report if present.	Employer access. Reevaluate invokes AI and remains gated.
Shared `/api/public/interviews/{token}/ | /start|answer|finish/`	Token-authenticated candidate session; answer payload is the shared turn contract.	InterviewTurn data and technical score/report after valid completion.	AI GATED: technical question generation and code/answer scoring require hooks and approved rubric. No fake technical score.

Data relationships

Same InterviewSession and InterviewTurn tables; technical type discriminator and module-specific evaluation fields are not separate resources.

Required verification

Assert technical kind round-trip through create/list/public session; shared lifecycle permissions; technical rubric and scores are validated only with deterministic fake AI; ensure screening routes remain compatible.

Acceptance gate: Complete shared non-AI lifecycle support and contract tests. Technical session content, scoring and report remain AI GATED; this module is not globally complete while gated actions remain.

MODULE 09 / SCOPED COMPLETE CANDIDATE | AI GATED REPORT GENERATION

Evaluation, ranking and HR decisions

Connect configurable weights, candidate comparisons, stored evaluation, HR decisions and PDF export. LLM-generated narrative/report creation and reevaluation remain gated.

Existing screen and workflow mapping

CompanyEvaluation and CompanyCandidateProfile score summaries, comparison selection, decision controls and report export.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET/PUT `/api/jobs/{job_id}/evaluation-weights/`	Weights for CV/screening/technical stages and missing-stage policy as accepted by serializer.	Job evaluation settings persisted to JobPosting/config relation; normalized values returned.	Admin/HR manager only. Validate range/total and require explicit handling for missing stages; recruiter/viewer cannot alter.
GET `/api/jobs/{job_id}/evaluations/ | /compare/?applications={uuid,...}`	Ranking filters/page; compare uses selected application IDs.	Stored score breakdowns and comparison rows; absent stage is null with a missing status, not zero.	Company-owned job. Enforce max compare count, reject IDs from another job, stable rank ordering.
GET `/api/applications/{application_id}/evaluation/`	Application UUID.	CandidateEvaluation with stage values, weighted aggregate, missing-stage metadata and report state when persisted.	Company owner. Present data freshness and missing interview/CV stage explicitly.
POST `/api/applications/{application_id}/evaluation/generate/`	Generate request / source selection from EvaluationGenerate serializer.	Persisted evaluation/report state and asynchronous or completed result per backend behavior.	AI GATED for narrative/LLM output; wait for hooks, persist, retry and error semantics. Do not mark generated from empty response.
POST/GET `/api/applications/{application_id}/decision/ | /decisions/`	Decision: selected, rejected or on_hold; optional note (max 1000) and notify flag.	Append-only Decision history with actor/time/note; optional candidate email notification.	Admin/HR manager/recruiter may decide; decisions are append-only. Duplicate same decision may return 409; refresh history.
GET `/api/applications/{application_id}/evaluation/pdf/`	Application UUID; no JSON body.	Binary PDF with application/report; use Blob and content-disposition filename.	Company access. Return unavailable until report exists; validate no private cross-tenant export.

Data relationships

CandidateEvaluation references Application/JobPosting and stage outputs; Decision is append-only and references application, actor and notification side effect.

Required verification

Weights validation/authorization; missing-stage arithmetic; ranking/compare ownership and limits; report generation fake AI and failure state; append-only decision history, duplicate 409, notification; PDF bytes, filename and authorization.

Acceptance gate: Weights, stored results, comparison, decisions and existing PDF export may be scoped complete after tests. LLM narrative and generated report path remains AI GATED; no global module completion until outputs are real and persisted.

MODULE 10 / SCOPED INTEGRATION

Candidate tracking and private links

Connect candidate-safe application timeline, interview invite status, dashboard and lost-link recovery. The public lost-link flow has no screen in the current frontend.

Existing screen and workflow mapping

CandidateDashboard and CandidateApplications; add API-backed safe timeline in existing detail/report patterns. Public track and lost-link screens are missing and excluded from new screen design in this plan.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/candidate/dashboard/ | /api/candidate/applications/`	Applications list supports status/page filters.	Candidate-safe counts, application references, job/company, stage and timeline. Paginated result envelope for list.	Candidate only; query is always current authenticated user. Never expose score, CV analysis, internal notes or HR decision note.
GET `/api/candidate/applications/{application_id}/`	Application UUID.	Safe detail/timeline projection derived from Module 4 Application + Module 7 session state.	Candidate owner; cross-user application IDs return no data.
POST `/api/candidate/applications/{application_id}/invitation/link/`	Application UUID; authenticated candidate requests private interview link.	New link/token, kind and expiry; return only to owning candidate.	Candidate owner; rate-limit and invalidate/rotate older links as backend specifies.
GET/POST `/api/public/track/{token}/ | /invitation/link/`	Opaque tracking token; link recovery request has minimal proof/context.	Candidate-safe tracking projection or renewed invitation URL.	Anonymous token is secret and rate-limited; no scores, private notes or other applicants. Avoid referrer/log leakage.
POST `/api/public/track/request-link/`	JSON {email}; response deliberately identical whether matching application exists.	Neutral anti-enumeration success response; email delivery uses configured backend.	Public; per-IP/email rate limits. Validate timing/content do not leak applicant existence.

Data relationships

No independent tracking aggregate: projections read Module 4 Application and Module 7 InterviewSession; tracking/invitation tokens are capabilities, not identifiers to persist in browser storage.

Required verification

Candidate ownership and safe-field allowlist; timeline stage transitions; invited-interview linking; token expiry/rotation; email anti-enumeration, rate limits and locmem assertions; no score/note leakage.

Acceptance gate: Authenticated candidate dashboard/list/detail are integrated. Public tracking/lost-link UI absence is a release gap; screen/API integration cannot be called end-to-end complete until that route is implemented under a separately approved UI task.

MODULE 11 / SCOPED COMPLETE CANDIDATE | AI GATED PRACTICE

Candidate mock interview

Integrate role catalog, session history, progress and session metadata. Starting questions, answer analysis, feedback and feedback retry rely on AI hooks and remain gated.

Existing screen and workflow mapping

InterviewSetup, InterviewRoom, InterviewReport, CandidateReports and history. Current question bank and deterministic local scoring are explicitly not backend truth.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/mock-interviews/roles/ | /progress/`	Optional role filters.	InterviewRole catalog; progress counters/level summary from persisted candidate sessions.	Candidate authenticated. Empty state is distinct from loading/error; use backend level progression.
GET/POST `/api/mock-interviews/`	Create JSON: interview_type, role_id?, use_cv?, question_count?; CV use requires uploaded resume/extracted text.	Paginated history and new MockInterviewSession linked one-to-one with shared InterviewSession; return UUID/status/type/role/timestamps.	Candidate owner. Validate type/role/limits and availability of CV; creation metadata can be tested before AI start.
GET/DELETE `/api/mock-interviews/{session_id}/`	Session UUID.	Session detail and history; deletion allowed only for valid states per backend policy.	Owner only. Enforce state transition and preserve required audit/history rules.
POST `/api/mock-interviews/{session_id}/start|answer|end/`	Start; answer payload `{answer: ...}` plus question/turn identity per schema; end finalizes.	Questions, InterviewTurn answer/score and final state/report.	AI GATED for question generation, scoring and feedback. Resume-based prompts additionally gated on Module 6 extraction.
POST `/api/mock-interviews/{session_id}/feedback/retry/`	Session UUID; no local-only recalculation.	Persisted regenerated feedback/version or explicit failed state.	AI GATED. Idempotency, cost/rate controls and retry cap must be tested before exposure.

Data relationships

InterviewRole; MockInterviewSession one-to-one InterviewSession; role FK and shared InterviewTurn. Candidate ownership enforced across history and operations.

Required verification

Role list, progress and history pagination; create constraints; delete state; owner isolation; start/answer/end transitions; fake AI questions/answer scoring/feedback, timeout and retry versioning before enabling gated operations.

Acceptance gate: Role/history/progress/session metadata can be scoped complete. Start, answer, end feedback and retry remain AI GATED; replace local deterministic score before enabling real candidate reports.

MODULE 12 / SCOPED INTEGRATION

Communities

Replace seeded candidate/company/admin community stores with the existing role-specific APIs for discovery, membership, moderation, invitations and posts.

Existing screen and workflow mapping

CandidateCommunities, CompanyCommunity and AdminCommunities provide usable screens; complete lifecycle actions, filters, member search and server state transitions against API. These are not new screens.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/candidate/communities/ | /mine/ | /{community_id}/ | /{id}/feed/`	Discovery filters/search/page where supported.	Community detail, membership state and paginated feed/post responses.	Candidate role; public fields only. Feed ordering and blocked/archived visibility follow backend policy.
POST `/api/candidate/communities/{id}/join/ | /leave/`	Community UUID; join may create request or membership per policy.	Membership/join-request state; membership belongs to Community + User.	Candidate only. Duplicate join/leave should reconcile as idempotent or explicit conflict.
GET/POST `/api/candidate/community-invitations/ | /{invitation_id}/accept|decline/`	List filters/page; action by invitation ID.	Invitation status and membership update.	Candidate invitee only; expired/revoked invitation is explicit; emit notification side effects.
POST `/api/candidate/community-posts/{post_id}/respond/`	Response/reaction content fields per serializer.	CommunityPost response/reaction persisted for User.	Candidate member and eligible post only; validate duplicate/reaction rules.
GET/POST/PATCH/DELETE `/api/company/communities/ | /{community_id}/`	Create/edit fields include name, description, visibility/eligibility and rules as defined by serializer.	Company-owned Community; list is paginated and detail includes state/counts.	Approved active company; admin/HR manager for management. Enforce company ownership and immutable archived rules.
POST/GET/DELETE `/api/company/communities/{id}/archive/ | /members/ | /members/{user_id}/`	Archive action; member list/search/pagination; remove by candidate ID.	Community lifecycle and member rows; candidate membership belongs to community/user.	Company write roles. Archive/removal side effects and historic post visibility are backend-defined.
GET/POST/DELETE `/api/company/communities/{id}/invitations/ | /{invitation_id}/`	Invite candidate IDs/emails and optional message; revoke by invitation ID.	Community invitation state and expiry.	Company community owner; reject ineligible/duplicate users; notifications use local email in test.
GET/POST `/api/company/communities/{id}/join-requests/ | /{request_id}/approve|reject/`	List filters/page; approve/reject action with optional reason.	Join request transition and membership outcome.	Company write roles. Validate still-pending state and prevent duplicate membership.
GET/POST/PATCH/DELETE `/api/company/communities/{id}/posts/ | /{post_id}/ | /{post_id}/responses/`	Post content, optional linked job, edit/archive and response fields.	CommunityPost linked to Community and optionally Job; responses linked to post/User.	Company owner for create/edit; candidates may read/respond if members. Generic OpenAPI response objects need serializer-level contract tests.
GET `/api/platform-admin/communities/ | /{community_id}/`	Monitor filters/page.	Read-only cross-company community summary/detail.	Platform admin only. Admin monitor is not community management by default.

Data relationships

Community belongs to Company; membership, invitation and join request reference Community and User; posts belong to Community and optional Job; post responses reference post/User.

Required verification

Candidate vs company vs admin permissions; company isolation; discovery/member filters and page; join/invite/approve/reject/archive transitions; post CRUD/responses; notification/email side effects; generic OpenAPI response contracts.

> Acceptance gate: Existing screens read/write server truth and handle every state; seeded local community data is retired after cutover. Track generic schema typing gaps and add request/response assertions before module acceptance.

MODULE 13 / SCOPED INTEGRATION | AI CONTROLS DEFERRED

Platform administration (non-AI scope)

Connect platform dashboard, users, company review, job/community monitoring, moderation, activity audit, health and alert acknowledgement. AI settings, prompt versions and AI usage are deliberately excluded.

Existing screen and workflow mapping

AdminDashboard, AdminUsers, AdminCompanies, AdminCommunities, AdminManagement/audit, AdminOperations and AdminReports. AdminSettings fields with no API must remain clearly non-persistent or disabled.

Endpoint contract map

METHOD / PATH	REQUEST / QUERY	RESPONSE / DATA	ACCESS / BEHAVIOR
GET `/api/platform-admin/dashboard/ | /health/ | /alerts/`	Optional time/filter query according to schema.	Dashboard aggregates, system health checks, alert list and severity/state.	Platform admin only; do not promise SLA/time-series not returned by API.
POST `/api/platform-admin/alerts/{alert_id}/acknowledge/`	Alert UUID and optional acknowledgement note.	Acknowledged state, actor and timestamp.	Platform admin; idempotent or explicit state conflict, with audit event.
GET/POST `/api/platform-admin/users/ | /{user_id}/detail|suspend|reactivate/`	List filters/page; action reason if required.	User identity, role, state, company summary; suspend/reactivate returns new status and audit.	Platform admin only. No password/token data; protect last active administrator and prevent self-lockout.
GET `/api/platform-admin/companies/ | /{company_id}/ | /onboarding/`	Search, approval/status and page filters.	Company, owner/members, onboarding documents and review state.	Platform admin only; file metadata private; action routes mapped in Module 2.
GET `/api/platform-admin/jobs/ | /{job_id}/ | /communities/ | /communities/{id}/`	Status/company/search/page filters where available.	Read-only operational records with owning company and relevant counts.	Platform admin only. Monitor-only screen does not imply global mutation rights.
GET `/api/platform-admin/flags/ | /moderation/`	Queue filters include status/type/severity/page as implemented.	ModerationFlag queue with source object, reporter, reason, status and timestamps.	Platform admin only; return safe source projection; cross-tenant moderation visibility is audited.
POST `/api/platform-admin/moderation/{flag_id}/review|resolve/ | /activity/{event_id}/flag/`	Review/resolve decision, reason and outcome fields per serializer.	Flag state transitions and linked audit event; activity flag creates moderation signal.	Platform admin only. Transition validation and immutable prior activity are required.
GET `/api/platform-admin/activity/ | /{event_id}/ | /activity/export/`	Actor/action/object/date filters, page; export is binary.	ActivityEvent detail and export rows; export content type/filename from backend.	Platform admin. Audit log is read-only except flag action; prevent unauthorized PII export.
POST `/api/reports/ | /api/public/report-job/`	Report job payload and public report submission contract are schema-defined.	Backend report job/status or public report record; no matching FE report-generation page is asserted.	Role/rate limits per endpoint. Treat as backend-only until a screen/workflow is approved.

Data relationships

ModerationFlag and BackupRecord plus shared ActivityEvent, SystemAlert and OutboundEmail. Platform operations are admin-only; company and community objects remain owned by their source tenants.

Required verification

Dashboard/health read permissions; user and company action state machine; flag review/resolve and audit append-only; export bytes/filters; alert acknowledgement; data isolation; admin MFA/session policy if configured.

> Acceptance gate: Non-AI operational routes may be scoped complete after tests. Exclude GET/PUT /platform-admin/ai-settings/, /prompts/ and /ai-usage/ from implementation. General platform settings and time-series analytics have no complete API; do not fake persistence.

Gap Register and Scope Boundaries

These are verified mismatches or missing contracts. The integration task must not imply that a UI or API exists when it does not; new screen design is outside this plan.

Gap	Current evidence	Plan treatment
Public job apply UI	Backend has GET/POST /api/public/jobs/{form_slug}/ and /apply/; current frontend routes do not include a public application form.	API contract/testing is included under Module 4; screen remains a gap and must be approved as a separate UI task.
Staff invitation acceptance UI	Backend exposes public invitation token lookup/accept; frontend has no public acceptance route.	Company invitation lifecycle can integrate; end-to-end acceptance cannot be called complete until a screen exists.
Public lost-tracking-link UI	Backend exposes /api/public/track/request-link/ and token view/link routes; no public recovery screen exists.	Authenticated candidate tracking can integrate; anonymous recovery is a tracked screen gap.
Company analytics	CompanyAnalytics is local-derived; no dedicated company analytics API contract is mounted.	Do not represent local aggregate as backend analytics or persist filters/results against a nonexistent endpoint.
Notification preferences	Candidate/company settings are local; notifications API covers inbox/read state, not preference updates.	Keep preference UI local-only or disable it; do not claim synchronization.
General platform settings	AdminSettings has local options; no general platform settings endpoint is mounted.	Exclude persistence claims; Module 13 settings controls with no contract remain deferred.
Platform reports	Dashboard/activity/health exist; broad report generation is limited/backend-only and lacks a matching FE workflow.	Integrate existing report surfaces only when their response contract fits; do not design new reports here.
Generic OpenAPI objects	Several community/admin/custom action responses are typed as generic object or omit useful response schema.	Use serializer/view output, add response contract tests, and improve schema docs as part of the owning module.
AI hooks and provider access	Interview hook placeholders use bank/fallback content and empty scoring; CV/report paths depend on provider behavior.	Gate Modules 6, 7, 8, 9 and 11 AI actions until deterministic fake-AI tests and provider access are approved.

Module 8 is not a separately wired API: its technical interview is a kind on Module 7 shared session endpoints. Module 13 AI settings, prompts and usage endpoints are intentionally out of scope regardless of current backend availability.

Test Strategy and Acceptance Gates

Build the frontend test harness as part of the integration foundation. Each module adds focused component, service and browser coverage, while backend pytest validates the persisted and authorized contract.

Frontend verification stack

Layer	Tooling and acceptance
Component	Vitest + React Testing Library: render real route/component states; assert accessible labels, user actions, error messages and role-visible controls.
API contract	MSW intercepts network requests: exact method/path/query/body, auth cookies, pagination, validation errors, 401 refresh, 403/404/409, multipart and binary responses.
Browser workflow	Playwright: login/refresh, company job lifecycle, application review, notification read, candidate tracking, community interactions and admin moderation. Missing public UI gaps are explicit failing/blocked acceptance items, not skipped silently.
Quality bar	Vite build and lint pass; no browser console errors; no unhandled network calls; no local demo entity mutation after API cutover; all error/loading/empty states exercised.

Backend and cross-cutting verification

- Use pytest with isolated database transactions/fixtures; fake AI adapters for gated module tests; Celery eager mode; locmem email. Production SMTP and external AI credentials are out of plan.
- For each operation test valid/invalid payloads, authentication, role permissions, ownership isolation, status transitions, pagination, file type/size, rate limits and DB persistence.
- Test side effects and rollback: notification/activity writes, email outbox, exports, token rotation/blacklisting and concurrent duplicate requests.
- Run regression on previously approved modules after each shared client/auth change and before release; include migration state and API schema diff review.

Measurable module acceptance

Criterion	Pass condition
Contract coverage	Every frontend service operation used by the module has request and response tests with success and representative 4xx/5xx cases.
Persistence	Create/update workflows survive reload and a fresh authenticated browser context; list/detail agree with database state.
Authorization	Candidate, company role, tenant isolation and platform-admin rules pass both backend tests and direct-route UI checks.
UX states	Every primary list/form/detail has loading, empty, success, validation, unauthorized, forbidden, not-found, conflict and retry behavior where applicable.
Regression	FE build/lint, module suite and previous module suites pass; no new console errors or unhandled API calls.
AI gate	For gated operations: fake-AI contract, deterministic output validation, timeout/retry/error persistence and provider credential readiness are reviewed before enabling.

Module Status Report and Approval Protocol

A module is not accepted by code completion alone. Record verification evidence, unresolved risks and explicit user approval before beginning the next module in sequence.

Required status record

Field	Record for each module
Scope and UI	Module number; existing screens touched; in-scope workflow; endpoint methods/paths; excluded or missing screens/APIs.
Implementation	API service files, components/stores changed, data migration/cutover behavior, feature flags and environment configuration.
Tests	Unit/component, API contract, backend/DB, functional/browser, security/permission and regression result; include command, counts and failures.
Data and security	Model relationships validated; UUID migration; tenant/owner checks; file/token handling; no PII/secrets in logs.
AI status	SCOPED COMPLETE vs AI GATED, the exact operation gated, hook/provider readiness, fake-AI evidence and remaining review.
Decision	Outstanding issues, explicit approver, approval date and next module authorized. Without approval, stop at the current module.

First-module recommendation

Begin Module 1 only after the shared client contract and refresh-cookie behavior are agreed with backend maintainers. The repository currently has no Vite proxy, no frontend API test harness, and no consumer of apiClient; starting with notifications would leave authentication and identity unresolved.

> Completion is module-scoped, not project-wide. Modules 6, 7, 8, 9 and 11 remain globally incomplete while any declared AI workflow is gated. A missing public screen or backend contract also prevents end-to-end completion of that workflow.

Source map

Frontend: frontend/src/api/client.js, authApi.js, store/authStore.js, candidateStore.js, companyStore.js, communityStore.js, platformStore.js, auditStore.js, frontend/src/config/routes.js, frontend/vite.config.js, frontend/package.json and role page files under frontend/src/pages/. Backend: backend/config/urls.py; each apps/Module*_*/urls.py, serializers.py, models.py, permissions.py and views; backend config/settings/local.py and test.py; generated GET /api/schema/?format=json.

Contract references

Use current backend OpenAPI for typed routes and schemas. Treat serializer/view behavior and regression tests as authoritative where the schema is generic or incomplete. Frontend test framework references selected for this plan: Vitest, React Testing Library, MSW and Playwright official documentation.

Planning boundary

This document is the integration blueprint only. It does not create frontend or backend services, alter application code, add screens, implement auth-cookie behavior, or start any module integration. Before implementation, re-check the schema and source at the then-current commit.